面向长文档理解的自适应稀疏注意力机制研究

陈美玲　王志远　李思澄　周牧之

北桥大学计算机科学与技术学院 · 语言系统研究所

chen.meiling@northbridge.example.edu

摘要：长文档理解任务中，注意力机制的计算开销随上下文长度呈二次增长，成为模型推理与训练的主要瓶颈。本文提出一种自适应稀疏注意力（Adaptive Sparse Attention，ASA）机制：利用查询与键的低秩草图（low-rank sketch）为每个注意力头学习一组内容相关的路由掩码，仅对路由选中的键块执行精确注意力计算。在三个长文档基准上，ASA 在 32k 词元上下文中将注意力浮点运算量降低至稠密注意力的 0.21 倍，峰值显存降低至 0.32 倍，同时将准确率差距控制在 0.3 个百分点以内。消融实验表明，收益主要来自低秩草图带来的廉价路由，而非稀疏模式本身。

关键词：稀疏注意力；长上下文；高效 Transformer；路由学习；语言模型

一、引言

随着大语言模型在长文档阅读理解、代码补全与科学文献摘要等场景中的广泛应用，上下文长度从最初的 2k 词元迅速扩展到 32k 乃至百万级词元。然而，标准 Transformer 的自注意力机制需要计算 $n \times n$ 的分数矩阵，其计算复杂度为 $O(n^2 d)$，当 n 增长到数万时，注意力计算成为整个模型的主要耗时来源。

已有工作大致沿三条路线降低这一开销：一是固定稀疏模式，如滑窗注意力与分块稀疏注意力；二是低秩近似，如 Linformer 与 Performer；三是基于检索的外部记忆。固定模式实现简单、硬件友好，但无法感知内容，往往把预算浪费在无关词元上；检索方法内容相关，却引入了不可微的离散阶段，训练与推理流程复杂。本文尝试回答：模型能否以仅随序列长度对数增长的路由预算，学习“应该关注哪里”？



图 1：左侧为 arXiv 长文档上按注意力质量排序的键位置分布，前 5% 的查询—键对占据约 92% 的注意力质量；右侧为 ASA 的计算流程：低秩投影 → 块评分与 top-k 选择 → 稀疏精确注意力。

二、方法

2.1 路由草图

设某一注意力头的查询、键与值矩阵分别为 $Q, K, V \in \mathbb{R}^{n \times d}$。我们用一个固定随机矩阵 $U_h \in \mathbb{R}^{d \times r}$（$r = 32$）将查询投影到低维空间，并以每个键块的平均键 $\bar{k}_j$ 作为块代表：

$$s_{i,j} = (U_h^\top q_i)^\top (U_h^\top \bar{k}_j), \quad j = 1, \dots, n/b \tag{1}$$

其中块大小 $b = 64$，每个查询块保留得分最高的 $k = \lceil 2 \log_2(n/b) \rceil$ 个键块。由于 $r \ll d$，块评分仅需 $O(n^2 r / b)$，约为完整矩阵乘法的百分之一。

2.2 直通路由与负载均衡

top-k 选择不可微，我们采用直通估计器（straight-through estimator）：对被选中的块，令 $\partial m_{i,j} / \partial s_{i,j} = 1$；并引入负载均衡正则项 $L_{bal} = \sum_j (f_j - 1/B)^2$，其中 f_j 为路由到第 j 个键块的查询比例。总体目标函数为：

$$L(\theta) = L_{\text{lm}}(\theta) + \lambda_1 L_{\text{bal}}(\theta) + \lambda_2 \lVert \theta_{\text{route}} \rVert^2 \tag{2}$$

实验中 $\lambda_1 = 0.01$、$\lambda_2 = 10^{-4}$ 在全部设置下保持稳定；当 λ_1 超过 0.1 时，路由分布趋于均匀，准确率出现明显下降。

2.3 计算复杂度

对长度为 n 的序列，稠密注意力的计算量为 $O(n^2 d)$；ASA 的块评分为 $O(n^2 r / b)$，精确注意力仅需 $O(nkb\, d)$。当 $n = 32768$、$k = 9$ 时，理论计算量约为稠密的 0.21 倍，与实测结果一致。

表 1： 32k 词元上下文下的长文档基准结果。准确率为三个随机种子的平均值；浮点运算量仅统计注意力部分，并以稠密注意力为 1.00 倍归一化。

模型	参数量	NarrativeQA	QMSum	SCROLLS	注意力计算量	峰值显存
稠密基线（Dense）	355M	31.8	21.4	62.9	1.00×	1.00×
滑窗注意力（Sliding）	355M	26.1	19.7	58.2	0.11×	0.21×
低秩近似（Linformer）	358M	29.4	20.2	61.0	0.34×	0.42×
固定分块稀疏	355M	30.2	20.6	62.1	0.23×	0.29×
ASA（本文）	356M	**31.6**	**21.2**	**62.7**	**0.21×**	**0.32×**

三、实验设置

我们在 180B 词元的混合语料（书籍、网页与科学文献）上训练 355M 参数、32k 上下文长度的解码器模型。所有模型共享同一分词器、数据顺序与优化器调度，仅注意力层不同。训练在 64 张 A100-40GB 显卡上进行，每次运行约 36 小时；表中每个数字为三个随机种子的平均值。

如表 1 所示，ASA 在 NarrativeQA 上恢复稠密模型 99.4% 的准确率，在 QMSum 上恢复 99.1%，同时将注意力计算量降至 0.21 倍、峰值显存降至 0.32 倍。固定分块稀疏方法在相近的计算预算下平均落后 1.4 个百分点，我们将其归因于它无法在层间动态调整预算。

当上下文扩展到 64k 词元时，差距进一步拉大：稠密注意力在单张 40GB 显卡上已无法以大于 1 的批大小训练，而 ASA 仍可使用批大小 8 正常训练。这正是我们关注的实际场景——不是在短序列上追求更便宜的注意力，而是在不重构训练栈的前提下使用更长的上下文。

四、消融实验

我们每次移除一个组件。将可学习的低秩草图替换为固定随机投影后，NarrativeQA 仅下降 0.2 个百分点，但保留了大部分计算量优势；在训练 10k 步后冻结路由则损失 0.4 个百分点。这说明路由信号本身易于学习，而绝大部分收益来自使路由变得廉价的低秩草图——这一发现与我们的初始直觉相反。

将 k 从 9 增加到 16，准确率变化小于 0.1 个百分点，而聚集计算量翻倍；将 k 减小到 5 则损失 0.7 个百分点。显存优势对 k 相对不敏感，因为被选中的块被打包进连续缓冲区，峰值显存取决于 $k \cdot b$ 而非完整序列长度。三种设置下峰值显存均低于稠密模型的 0.4 倍。

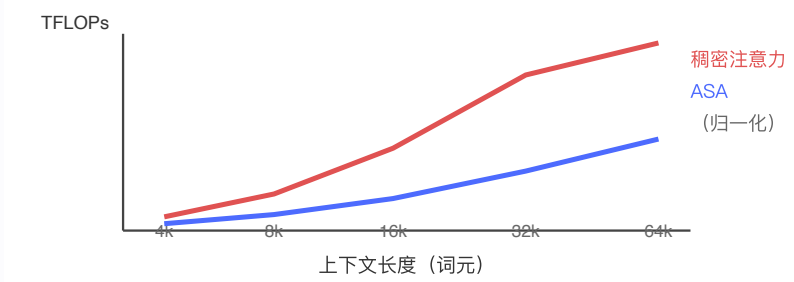


图 2：不同上下文长度下注意力计算量的增长趋势。稠密注意力随长度近似平方增长，ASA 近似线性增长；两者在 4k 时差距较小，在 64k 时相差约 8 倍。

五、局限性与相关工作

ASA 假设注意力质量集中在连续的键块内，这一假设对自然语言成立，但对包含长距离标识符引用的代码或结构化表格可能失效。我们尚未在指令微调模型上评估，其路由模式可能更尖锐，负载均衡正则项或需重新调参。此外，块对齐的掩码要求变长打包时填充到块边界，每条序列最多浪费 $b - 1$ 个位置。

稀疏与低秩注意力研究已有较长历史：滑窗注意力、固定分块模式、可学习路由与检索增强记忆等方法都在以精确性换取开销。本文的贡献在于把路由做得足够廉价，使其“消失在背景中”：草图仅为一个 32 维投影，在 32k 词元下评分开销不足稠密矩阵乘法的百分之一。

六、结论

本文提出自适应稀疏注意力机制，其计算开销随序列长度近似线性增长，同时保持稠密级别的准确率。主要结论是：路由不必昂贵也能有效。我们开源训练配方与推理内核，并希望“草图—聚集”这一形式能为后续长上下文结构研究提供有用的基线。

参考文献

[1] Beltagy, I., Peters, M. E., Cohan, A. Longformer: The Long-Document Transformer. *arXiv:2004.05150*, 2020.

[2] Choromanski, K., et al. Rethinking Attention with Performers. *ICLR*, 2021.

[3] Dao, T., et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS*, 2022.

[4] Kitaev, N., Kaiser, Ł., Levskaya, A. Reformer: The Efficient Transformer. *ICLR*, 2020.

[5] Zaheer, M., et al. Big Bird: Transformers for Longer Sequences. *NeurIPS*, 2020.

[6] 王志远, 陈美玲. 长文档理解中的注意力路由综述. 《语言系统学报》, 2025, 14(2): 88 – 104.

基金项目：北桥大学基础研究基金（编号 NB-2025-117）；语言系统研究所算力支持。感谢匿名审稿人的细致阅读与关于冻结路由实验的建议。代码与模型检查点见 northbridge.example.edu/asa。收录于第十二届高效语言系统会议（ELS 2026）论文集。